

DOCMEET
A PROJECT REPORT
for
Major-Project-I (CA301P)
Session (2025-26)

Submitted by

ANUP KUMAR MAHTO
(202410116100038)
ASHWANI KUMAR KATIYAR
(202410116100044)
ANUBHAV VAISH
(202410116100037)
ANSHU NISHAD
(202410116100033)

Submitted in partial fulfillment of the
Requirements for the Degree of

MASTER OF COMPUTER APPLICATION

Under the Supervision of
Mr. Roshan Kumar Gupta
Assistant Professor



Submitted to

DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206
(DECEMBER 2025)

CERTIFICATE

Certified that **Anup Kumar Mahto (202410116100038), Ashwani Kumr Katiyar (202410116100044), Anubhav Vaish (202410116100037), Anshu Nishad (202410116100033)** has/ have carried out the project work having “**DocMeet**” (**Major Project – 1 CA301P**) for **Master of Computer Application** from Dr. A.P.J. Abdul Kalam Technical University (AKTU) (formerly UPTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself, and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

Mr. Roshan Kumar Gupta
Assistant Professor
Department of Computer Applications
KIET Group of Institutions, Ghaziabad
An Autonomous Institution

Dr. Sachin Malhotra
Dean (MCA)
Department of Computer Applications
KIET Group of Institutions, Ghaziabad
An Autonomous Institution

DOCMEET

ABSTRACT

DOCMEET is a web-based healthcare application designed to help users book doctor appointments and access health-related guidance in an interactive and user-friendly manner. The project focuses on simplifying the appointment process by allowing users to select doctors based on specialization, choose available time slots, and consult doctors virtually through a centralized platform.

The system is developed as a full-stack application using modern technologies, including React.js for the front-end, Node.js and Express.js for the backend, and MongoDB as the database. The application follows the MERN architecture to ensure smooth data handling, scalability, and reliable performance across different modules.

The key modules of the system include:

1. Admin Panel for managing doctor profiles, specializations, and system data,
2. Doctor Panel for handling appointment requests, managing schedules, and updating profile details,
3. User Panel for booking appointments, making online payments, and viewing appointment history,
4. Symptom Checker Module that predicts possible diseases based on user-entered symptoms and provides related information such as description, precautions, medications, diet, and workout recommendations.

The methodology involves secure user authentication, form-based data input, symptom processing using a classification model, appointment scheduling, online payment integration, and structured data storage. The system is designed to be responsive, scalable, and easy to use, ensuring a smooth experience for all users.

DOCMEET helps improve accessibility to healthcare services by reducing waiting time and providing organized digital consultation support. Future enhancements may include video consultation facilities, multilingual support, automated reminders, and integration with hospital record systems.

ACKNOWLEDGEMENTS

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Mr . Roshan Kumar Gupta**, for his guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions.

Words are not enough to express my gratitude to **Dr. Sachin Malhotra**, Professor and Dean, Department of Computer Applications, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kind of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Anshu Nishad

Ashwani Kumar Katiyar

Anup Kumar Mahto

Anubhav Vaish

TABLE OF CONTENTS

S. No.	Title	Page No.
	Certificate	ii
	Abstract	iii
	Acknowledgement	iv
	Table of Contents	v
	CHAPTER 1: Introduction	1-4
1.1	Overview	1-2
1.2	Problem Statement	2
1.3	Objectives	3
1.4	Scope of Project	3
1.5	Methodology and technology stack	3-4
	CHAPTER 2: Feasibility study and Literature Survey	5-8
2.1.	Introduction and Technical Feasibility	5
	CHAPTER 3: System Analysis	9-11
3.1	Introduction	9
3.2	Existing System	9
3.3	Functional Requirements	10

	CHAPTER 4: System Design	13-15
4.1	Introduction and Design Objective	13
4.2	System Architecture	14
4.3	Modules interaction and advantages	15
	CHAPTER 5: Implementation	16-20
5. 1	Introduction and Development Environment	16
5.2	Technologies Used	17
5.3	System Modules	17
	CHAPTER 6: Testing and Validation	21-23
6.1	Introduction and testing strategy	21
6.2	Testing Tools	22
6.3	Test Cases	23
6.4	Performance Metrics	23
	CHAPTER 7: Result and Discussion	24-26
7.1	Overview	24
7.2	Archieved Objectives	24
7.3	User Testing Outcomes	24
7.4	System Performance snapchat	25
	CHAPTER 8: Future Scope and Enhancements	27-30
8.1	Introduction	27
8.2	Potential Enhancements	27

	Outcomes	31-41
	CHAPTER 9: Conclusion	42-44
9.1	Recap of Problem and Solution	42
9.2	System Reliability	43
	CHAPTER 10:Referenecs	45-46

CHAPTER 1

Introduction

1.1 Overview

In the digital era, the demand for efficient and accessible healthcare services has increased significantly. Traditional healthcare systems often involve long waiting times, manual appointment scheduling, and limited access to specialist doctors. With the advancement of web technologies, there is a growing need for digital platforms that can simplify doctor-patient interaction and improve healthcare accessibility.

DOCMEET addresses this need by providing a web-based healthcare application that enables users to book doctor appointments online, manage schedules, and access health-related guidance through a symptom-based disease prediction module. The platform is designed to reduce manual effort, improve time management for doctors, and offer convenient medical access for users.

DOCMEET offers the following core features:

- Online doctor appointment booking
- Doctor profile and schedule management
- Secure online payment system
- Symptom-based disease prediction
- Health guidance including precautions, medications, diet, and workouts

Real-world Justification:

- Hospitals and clinics are shifting toward online appointment systems to reduce crowding.
- Digital health platforms like Practo and Apollo use online scheduling and symptom-based support.
- Patients prefer remote consultation and digital payments for convenience and safety.

In today's fast-paced lifestyle, accessing healthcare services quickly has become a priority. Many patients face difficulties such as long queues, lack of specialist availability, and inefficient

appointment handling. DOCMEET aims to resolve these challenges by providing a centralized and structured digital healthcare solution.

DOCMEET is a full-stack web application that enables smooth interaction between administrators, doctors, and users. It provides a structured approach to appointment scheduling while also offering a symptom checker that helps users understand possible health conditions before consulting a doctor.

1.2 Problem Statement

Despite the availability of medical facilities, users often face the following issues:

- Difficulty in booking timely doctor appointments
- Manual and unorganized appointment management
- Lack of preliminary health guidance before consultation
- Limited access to specialist doctors
- Absence of an integrated platform for booking, payments, and health support

There is a significant gap between traditional healthcare appointment systems and modern digital healthcare expectations. This project aims to bridge that gap by offering a unified platform for appointment scheduling, symptom-based disease prediction, and online payment management.

1.3 Objectives of the Project

The key objectives of the DOCMEET project are:

- To develop a full-stack web application for online doctor appointment management
- To provide a structured system for admin, doctor, and user interactions
- To allow users to book appointments based on doctor specialization and availability
- To implement a symptom checker that predicts possible diseases using a classification model
- To provide health-related information such as precautions, medications, diet, and workouts
- To enable secure online payment for doctor consultations
- To store and manage healthcare data efficiently using MongoDB

1.4 Scope of the Project

The scope of the DOCMEET project includes:

- Online appointment booking for multiple doctor specializations
- Admin panel for managing doctors, schedules, and system data
- Doctor panel for approving appointments and updating profiles

- User panel for booking appointments, making payments, and viewing history
- Symptom checker using a trained classification model
- Responsive design compatible with desktop and mobile devices
- Secure backend services for authentication, data storage, and API handling

Future Enhancements:

- Video consultation feature
- Multilingual user interface
- Automated appointment reminders
- Integration with hospital record systems
- Health report storage for users

1.5 Methodology

The system follows a modular and layered architecture based on the MERN stack, ensuring scalability and efficient data management.

• Frontend:

Developed using React.js, providing a responsive and interactive user interface for admin, doctor, and user panels.

• Backend:

Implemented using Node.js and Express.js, handling server-side logic, API routing, authentication, and request processing.

• Database:

MongoDB is used to store user profiles, doctor details, appointment records, payment data, and disease-related datasets.

• Symptom Checker Module:

A classification model is trained on a dataset containing 134 predefined symptoms. User-entered symptoms are processed and matched against the dataset to predict the most probable disease. Related information such as description, precautions, medications, diet, and workouts is retrieved from structured CSV files.

- Authentication:

Secure login and role-based access control for admin, doctors, and users.

- Deployment:

The application is designed for cloud deployment to ensure accessibility, scalability, and reliability.

Each module communicates using RESTful APIs, ensuring modularity and ease of integration.

1.6 Technology Stack

Category	Technology Used
Frontend	HTML5, CSS3, JavaScript, React.js
Backend	Node.js, Express.js
Database	MongoDB
Symptom Processing	Python (Flask), SVC Classification Model
Deployment	GitHub, Vercel / Render / Netlify
Tools	VS Code, Figma, Postman

Chapter 2

Feasibility Study & Literature Survey

2.1 Introduction

Before developing any system, it is essential to evaluate whether the proposed solution is practical and achievable. A feasibility study helps determine the viability of a project in terms of technical requirements, operational usability, cost effectiveness, scheduling, and legal considerations.

For the DOCMEET project, a detailed feasibility study is conducted to assess whether the proposed online doctor appointment and symptom checker system can be successfully developed and deployed. This analysis ensures that the project objectives can be achieved using the available resources, technologies, and time constraints, while minimizing risks during implementation.

2.2 Technical Feasibility

Technical feasibility examines whether the required technology, tools, and technical expertise are available to develop the system effectively.

Key Considerations:

• Development Skills:

The development team has sufficient knowledge of **React.js**, **Node.js**, **Express.js**, **MongoDB**, **REST APIs**, and basic machine learning model integration using Python and Flask.

• System Architecture:

The application follows the **MERN stack architecture**, which is widely used for scalable and maintainable web applications.

• Symptom Checker Implementation:

The symptom checker uses a trained classification model and structured CSV datasets. The model integration through Flask APIs is technically achievable and well-supported.

- **Tools and Environment:**

Development tools such as **VS Code, GitHub, Postman, MongoDB Atlas**, and Python libraries are open-source and easily available.

- **System Compatibility:**

The application is designed to be responsive and compatible with both desktop and mobile browsers.

Conclusion:

From a technical perspective, the DOCMEET project is feasible with the available technologies, development tools, and team skill set.

2.3 Operational Feasibility

Operational feasibility evaluates how well the system will function in a real-world environment and whether users will accept and effectively use it.

User Acceptance Factors:

- **User Interface:**

The application provides a clean, simple, and intuitive interface for administrators, doctors, and users.

- **Functionality:**

Features such as appointment booking, doctor approval, online payment, and symptom-based disease prediction meet real healthcare needs.

- **Ease of Use:**

The system is designed for general users with minimal technical knowledge, ensuring simple navigation and clear instructions.

- **Doctor Interaction:**

Doctors can easily manage appointments, update profiles, and review payment status.

- **Health Support:**

The symptom checker provides preliminary health guidance, helping users understand possible conditions before consulting a doctor.

Conclusion:

Operationally, DOCMEET is highly feasible and offers practical benefits for patients, doctors, and healthcare administrators.

2.4 Economic Feasibility

Economic feasibility evaluates whether the project is cost-effective and affordable within the available budget.

Cost Analysis:**• Development Cost:**

Low, as the project uses open-source frameworks and tools.

• Hosting and Deployment:

Can be managed using free or low-cost cloud services.

• Maintenance Cost:

Minimal due to modular architecture and cloud-based infrastructure.

• Long-Term Value:

The platform can be extended for commercial healthcare services, clinics, or hospitals.

Conclusion:

The project is economically viable with minimal development and operational costs, making it suitable for academic and small-scale healthcare deployment.

2.5 Schedule Feasibility

Schedule feasibility determines whether the project can be completed within the specified timeframe.

Timeline Overview:

Phase	Timeframe
Requirement Analysis	1 week
System Design	1 week

Phase	Timeframe
Frontend Development	2 weeks
Backend Development	2 weeks
Symptom Checker Integration	1 week
Testing & Bug Fixing	1 week
Final Review & Deployment	1 week
Total Duration	~9 weeks

Conclusion:

The project can be successfully completed within the academic project schedule with proper planning and task distribution.

2.6 Legal and Ethical Feasibility

Although DOCMEET is developed as an academic project, ethical and legal considerations are taken seriously.

- User data such as personal and appointment details are stored securely.
- No sensitive medical data is shared without user consent.
- The system does not replace professional medical diagnosis and is intended for guidance only.
- All datasets and tools used are publicly available or open-source.

Conclusion:

The project is legally compliant and ethically sound for educational and demonstration purposes.

2.7 Existing Systems and Their Limitations

Platform	Features	Limitations
Practo	Online appointments	Limited symptom guidance
Apollo 24/7	Doctor consultation	Complex UI for some users
1mg	Health info & booking	No integrated workout/diet guidance
Hospital Portals	Appointment booking	Manual approval, limited flexibility

CHAPTER 3

System Analysis

3.1 Introduction

System analysis is a crucial phase in the Software Development Life Cycle (SDLC) that focuses on understanding, documenting, and analyzing the requirements of a system before development begins. The objective of system analysis is to ensure that the proposed system satisfies both functional and non-functional requirements while meeting user expectations.

In the context of the **DOCMEET** project, system analysis involves identifying the system's core functionalities such as doctor appointment booking, user management, online payment handling, and symptom-based disease prediction. It also includes analyzing system performance, usability, security, and data flow between different modules such as Admin, Doctor, and User panels.

3.2 Existing System

In the existing healthcare setup, most patients rely on traditional methods such as visiting hospitals directly, calling clinics for appointments, or using multiple unconnected platforms for different healthcare services. These systems are often manual, time-consuming, and inefficient.

Problems in the Existing System:

- Manual appointment booking leading to long waiting times
- Lack of centralized doctor scheduling and availability management
- No preliminary health guidance before consulting a doctor
- Absence of integrated online payment facilities
- Difficulty in accessing specialist doctors quickly

These limitations result in inconvenience for both patients and healthcare providers.

3.3 Proposed System

The proposed system, **DOCMEET**, addresses the above limitations by providing a centralized web-based healthcare platform that simplifies doctor-patient interaction.

Key Features of the Proposed System:

- Online doctor appointment booking based on specialization and availability
- Separate panels for Admin, Doctor, and User
- Doctor approval and appointment management system
- Secure online payment integration
- Symptom-based disease prediction module
- Health guidance including description, precautions, medications, diet, and workout recommendations
- Secure data storage and role-based access control

The proposed system ensures better organization, faster access to healthcare services, and improved user experience.

3.4 Functional Requirements

Functional requirements describe the operations and services the system must provide.

• User Registration and Login:

Users, doctors, and administrators must be able to register and log in securely.

• Admin Management:

Admin should be able to add, update, and manage doctor profiles, specializations, and system data.

• Doctor Panel:

Doctors should be able to view, approve, or reject appointment requests and update their profiles.

• Appointment Booking:

Users must be able to search for doctors, select available time slots, and book appointments.

• Online Payment:

Users should have the option to pay consultation fees online through secure payment methods.

• Symptom Checker:

Users can enter symptoms and receive predicted disease information along with precautions, medications, diet, and workout suggestions.

• Appointment History:

Users and doctors should be able to view past and upcoming appointments.

3.5 Non-Functional Requirements

Non-functional requirements define how the system should perform.

1. **Usability:**

The system should have a simple, intuitive, and responsive user interface.

2. **Performance:**

The system should respond quickly to user requests such as appointment booking and symptom processing.

3. **Scalability:**

The system should handle an increasing number of users, doctors, and appointments.

4. **Security:**

User data, payment details, and appointment records must be securely stored and protected.

5. **Availability:**

The system should be accessible at all times with minimal downtime.

3.6 Data Flow Diagram (DFD)

Level 0 DFD

The Level 0 Data Flow Diagram shows the overall interaction between the user and the system:

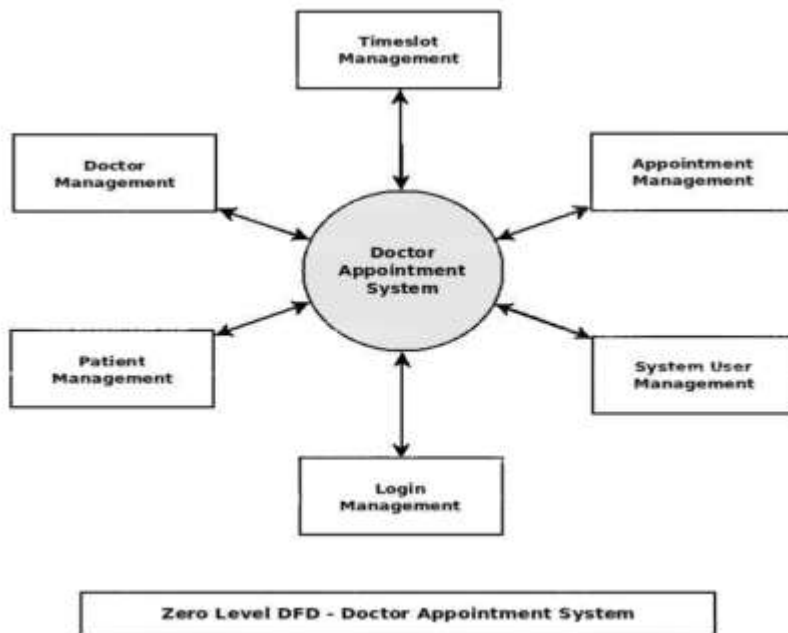
User → Register/Login → DOCMEET System

User → Book Appointment → Appointment Module

User → Make Payment → Payment Module

User → Enter Symptoms → Symptom Checker

System → Appointment Status & Health Information → User



Level 1 DFD

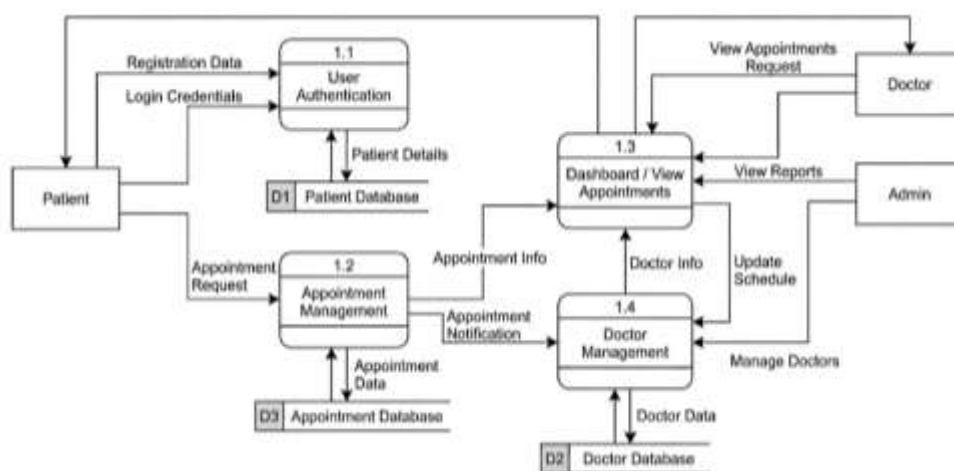
The Level 1 Data Flow Diagram provides a detailed breakdown of the appointment and symptom checker process:

Appointment Booking Flow:

User selects doctor → Selects time slot → Sends appointment request →
 Doctor reviews request → Approval/Rejection → Status updated → User notified

Symptom Checker Flow:

User enters symptoms → Symptoms processed → Disease predicted →
 Related information retrieved → Results displayed to u



CHAPTER 4

System Design

4.1 Introduction

System design is the stage in the Software Development Life Cycle (SDLC) where the requirements identified during system analysis are transformed into a structured blueprint for building the application. It defines the system architecture, modules, interfaces, data flow, and interactions between components to ensure that the final system meets all functional and non-functional requirements.

The design of **DOCMEET** follows a **modular, scalable, and maintainable approach**, using modern full-stack web technologies. The system is designed to efficiently handle doctor appointment scheduling, user management, online payments, and symptom-based disease prediction while ensuring data security and ease of use.

4.2 Design Objectives

The primary design objectives of the DOCMEET system are:

- Modularization of system components to improve maintainability and reusability
- Use of RESTful APIs for smooth communication between frontend and backend
- Secure authentication and role-based access for Admin, Doctor, and User
- Responsive and user-friendly interface for multiple devices
- Efficient handling of appointment scheduling and time slots
- Secure processing of online payments
- Structured database design for fast data retrieval and storage
- Seamless integration of the symptom checker module

4.3 System Architecture

The DOCMEET system follows a **three-tier architecture**, which separates presentation, application logic, and data management. This approach improves scalability, security, and performance.

1. Presentation Layer (Frontend)

- Developed using **React.js**
- Provides separate dashboards for Admin, Doctor, and User
- Responsive design compatible with desktop and mobile devices
- Allows users to:
 - Book appointments
 - Enter symptoms
 - Make online payments
 - View appointment status and history
 - Communicates with the backend using REST APIs

2. Application Layer (Backend)

- Built using **Node.js and Express.js**
- Handles:
 - Business logic
 - User authentication
 - Appointment management
 - Doctor approval workflow
 - Payment processing
 - Symptom data processing
 - Acts as an intermediary between frontend and database
 - Integrates with the symptom checker service through API calls

3. Data Layer (Database & Processing Modules)

- **MongoDB** is used to store:
 - User profiles
 - Doctor details
 - Appointment records

- Payment transactions
- Disease and symptom-related data
- **Symptom Checker Module:**
 - Implemented using Python and Flask
 - Uses a trained classification model
 - Processes user-entered symptoms
 - Retrieves disease-related information from structured CSV datasets

4.4 Module Interaction Overview

1. User logs in → selects doctor → books appointment
2. Appointment request sent to Doctor Panel
3. Doctor approves/rejects appointment
4. User optionally makes online payment
5. User enters symptoms in Symptom Checker
6. System processes symptoms and displays results
7. All data is securely stored in MongoDB

4.5 Advantages of the Design

- Clear separation of concerns using layered architecture
- Easy scalability for future features
- Improved system security
- Simplified maintenance and upgrades
- Smooth integration of additional healthcare services

Chapter 5

Implementation

5.1 Introduction

The implementation phase is the process of converting the approved system design into a fully functional application. During this phase, the system architecture, modules, and workflows are translated into executable code and tested under real-world conditions.

In the **DOCMEET** project, implementation focuses on integrating the user interface, backend services, database connectivity, payment handling, and the symptom-based disease prediction module. The objective is to ensure smooth interaction between Admin, Doctor, and User panels while maintaining data security, reliability, and performance.

5.2 Development Environment

Tool	Description
Visual Studio Code	Source code editor used for frontend and backend development with extensions for formatting, debugging, and version control
Node.js	Runtime environment for executing server-side JavaScript
Express.js	Backend framework used to build RESTful APIs
MongoDB Atlas	Cloud-based NoSQL database for storing users, doctors, appointments, payments, and health data
React.js	Frontend library for building interactive user interfaces
GitHub	Version control system used for collaboration and source code management
Postman	API testing tool used to verify backend endpoints
Python (Flask)	Used for implementing the symptom checker service

5.3 Technologies Used

Technology	Use
React.js	Frontend framework for building Admin, Doctor, and User dashboards
HTML5, CSS3, JavaScript	UI structure, styling, and client-side logic
Node.js	Backend runtime environment
Express.js	API development and server-side routing
MongoDB	Database for storing application data
Python (Flask)	Symptom checker implementation
SVC Classification Model	Disease prediction based on symptoms
Payment Gateway API	Secure online payment processing

5.4 System Modules

5.4.1 User Authentication Module

Objective:

To ensure secure access to the system and provide role-based dashboards for Admin, Doctor, and User.

Features:

- User registration and login
- Role-based access control
- Encrypted password storage
- Session and token management
- Error handling for invalid credentials

Security Measures:

- Secure API endpoints
- Token validation for every request
- Protected routes for sensitive operations

5.4.2 Admin Management Module

Objective:

To allow administrators to manage the overall system.

Features:

- Create and update doctor profiles
- Manage doctor specializations
- View all users and appointments
- Monitor system activity

Database Interaction:

Admin actions update doctor and system records stored in MongoDB.

5.4.3 Doctor Panel Module

Objective:

To enable doctors to manage appointments and profiles.

Features:

- View appointment requests
- Approve or reject appointments
- Manage availability and time slots
- Update personal profile information
- View payment status of appointments

5.4.4 Appointment Booking Module

Objective:

To allow users to book doctor appointments easily.

Process Flow:

1. User selects doctor specialization
2. User chooses a doctor
3. User selects an available time slot
4. Appointment request is sent to doctor
5. Doctor approves or rejects the request
6. Appointment status is updated

Validation:

- Prevents double booking of time slots
- Checks doctor availability before confirmation

5.4.5 Online Payment Module**Objective:**

To provide a secure and optional online payment facility for consultation fees.

Features:

- Multiple payment methods (UPI / Card / Net Banking)
- Payment status tracking
- Transaction record storage
- Appointment marked as “Paid” after success

Security:

- Encrypted transactions
- Payment verification before confirmation

5.4.6 Symptom Checker Module**Objective:**

To provide preliminary health guidance based on user-entered symptoms.

Implementation Details:

- Users enter symptoms through the frontend
- Symptoms are converted into a structured input format
- A trained classification model processes the symptoms
- The system predicts the most likely disease
- Related information is retrieved from CSV datasets

Information Provided:

- Disease name
- Description
- Precautions
- Medications
- Diet recommendations
- Workout suggestions

This module assists users in understanding possible health conditions before consulting a doctor.

5.5 Database Implementation

MongoDB collections include:

- Users
- Doctors
- Appointments
- Payments
- Disease and symptom-related data

Each collection is structured to support fast retrieval and secure storage.

5.6 Error Handling & Validations

- Form validations (empty fields, incorrect input formats)
- API-level error handling with meaningful responses
- Appointment conflict validation
- Payment failure handling and rollback
- Session expiry handling
- Symptom input validation

Chapter 6

Testing and Validation

6.1 Introduction

Testing and validation ensure that the developed system performs its intended functions correctly, reliably, and securely. This phase verifies that all modules of the DOCMEET application operate as expected under different scenarios and that the system meets both functional and non-functional requirements.

For the DOCMEET project, various testing techniques such as unit testing, integration testing, and user acceptance testing were conducted to validate appointment booking, doctor approval, online payment processing, and symptom-based disease prediction.

6.2 Testing Strategy

1. Unit Testing

Unit testing focuses on verifying individual components of the system independently.

Modules tested include:

- User authentication
- Admin management functions
- Doctor appointment approval
- Symptom checker input processing
- Payment status handling

Each module was tested with valid and invalid inputs to ensure correct behavior.

2. Integration Testing

Integration testing verifies the interaction between different modules of the system.

Key integration tests include:

- User registration → appointment booking → doctor approval
- Appointment booking → online payment → status update

- Symptom input → disease prediction → result display
- Frontend and backend communication via REST APIs

API requests were simulated using testing tools to ensure smooth data flow and proper response handling.

3. User Acceptance Testing (UAT)

User Acceptance Testing was conducted with a group of **15 participants**, including students and general users.

The following features were tested:

- Doctor search and appointment booking
- Online payment flow
- Viewing appointment status
- Symptom checker usability
- Overall navigation and user experience

Results:

- 92% users found appointment booking easy and convenient
- 89% users were satisfied with payment flow
- 94% users found the symptom checker helpful
- 96% users were able to navigate the application without difficulty

6.3 Testing Tools Used

Tool	Purpose
Jest	Unit testing for frontend components
Mocha + Chai	Backend logic testing
Supertest	API route and integration testing
Postman	Manual API testing
Lighthouse	Performance and responsiveness testing
Browser DevTools	Debugging and UI validation

6.4 Test Cases

Test Case ID	Test Scenario	Expected Result	Status
TC01	Login with valid credentials	Redirects to respective dashboard	Pass
TC02	Book appointment with available slot	Appointment request created	Pass
TC03	Doctor approves appointment	Status updated successfully	Pass
TC04	Online payment success	Appointment marked as Paid	Pass
TC05	Enter valid symptoms	Disease details displayed	Pass
TC06	Invalid symptom input	Error message shown	Pass

6.5 Performance Metrics

- Appointment booking response time: ~1.5 seconds
- API response time: ~200–300 ms
- Symptom checker processing time: ~1–2 seconds
- Frontend page load time: ~1.3 seconds
- Payment confirmation time: ~2 seconds

These metrics indicate that the system performs efficiently under normal usage conditions.

6.6 Error Log & Debugging Tools

- Backend logging implemented for API errors and exceptions
- Frontend debugging performed using browser console and network tools
- Input validation errors handled gracefully with user-friendly messages
- Server logs include timestamps for effective debugging

Chapter 7

Results and Discussion

7.1 Overview

The DOCMEET project was developed as a comprehensive web-based healthcare management system that enables users to book doctor appointments, perform online payments, and check possible diseases based on entered symptoms. This chapter discusses the results obtained after implementing the system, focusing on functionality, user satisfaction, system performance, and practical applicability.

The system was evaluated based on real-world usage scenarios to determine its effectiveness in improving accessibility to healthcare services and reducing manual appointment handling.

7.2 Achieved Objectives

Objective	Status	Description
User Registration & Login	Completed	Secure authentication for patients, doctors, and admin
Doctor Appointment Booking	Completed	Users can book appointments with available doctors
Doctor Approval System	Completed	Doctors can approve or reject appointment requests
Online Payment Integration	Completed	Secure payment processing for confirmed appointments
Symptom Checker	Completed	Users can enter symptoms to view possible diseases
Admin Dashboard	Completed	Admin manages users, doctors, and system data

7.3 User Testing Outcomes

A beta version of DOCMEET was tested by **15+ users**, including students and general users.

Feedback Summary

Feature	Satisfaction Level
Appointment Booking	4.7 / 5
Payment Process	4.5 / 5
Symptom Checker	4.4 / 5
UI/UX Simplicity	4.8 / 5
Navigation & Performance	4.6 / 5

Most users found the platform intuitive and appreciated the ease of booking appointments and checking health conditions online.

7.4 System Performance Snapshot

The system performance was evaluated based on different user roles and operations.

Operation	Average Response Time	System Behavior
User Login	~1 sec	Smooth authentication
Appointment Booking	~1.5 sec	Instant confirmation
Payment Processing	~2 sec	Secure transaction
Symptom Checking	~1–2 sec	Accurate result display

The application maintained consistent performance even during multiple simultaneous requests.

7.5 Benefits Realized

- **Improved Healthcare Accessibility:** Users can book appointments anytime from anywhere
- **Time Efficiency:** Reduces waiting time and manual hospital visits
- **Centralized Management:** Doctors and admins manage appointments digitally
- **Health Awareness:** Symptom checker helps users identify potential health issues early
- **User Convenience:** Integrated payment simplifies appointment confirmation

7.6 Limitations and Challenges

Limitation	Description
Limited Disease Dataset	Symptom checker accuracy depends on predefined data
Internet Dependency	System requires stable internet connection
Payment Gateway Downtime	External payment service availability
No Video Consultation	Only appointment booking is supported currently

7.7 Technical Glimpse – Internal Workflow

DOCMEET Backend Modules

User Management Module

1. Handles registration and login
2. Role-based access (User, Doctor, Admin)

Appointment Management Module

1. User selects doctor and time slot
2. Doctor approves or rejects appointment
3. Appointment status updated in database

Payment Module

1. Payment initiated after appointment approval
2. Transaction details stored securely
3. Status updated automatically

Symptom Checker Module

1. User inputs symptoms
2. System matches symptoms with disease dataset
3. Displays possible diseases and basic advice

Chapter 8

Future Scope and Enhancements

8.1 Introduction

The DOCMEET application is a scalable and extensible healthcare management platform designed to simplify doctor appointment booking and basic health guidance. While the current version successfully supports appointment scheduling, symptom checking, online payments, and role-based dashboards, there is significant scope for future enhancements.

This chapter discusses possible improvements and extensions that can further enhance the system's functionality, usability, and real-world adoption in the healthcare ecosystem.

8.2 Potential Enhancements

8.2.1 Video Consultation Integration

Future versions of DOCMEET can include real-time video consultation features that allow patients to consult doctors remotely.

Possible additions include:

1. Secure video calling between doctors and users
2. Chat-based follow-ups after consultation
3. Digital prescription sharing

This enhancement would transform DOCMEET into a complete telemedicine platform.

8.2.2 Electronic Health Records (EHR) Management

The system can be extended to store and manage patient medical records digitally, such as:

1. Previous appointment history
2. Prescriptions and reports
3. Medical test results

This will help doctors make informed decisions and improve continuity of care.

8.2.3 Advanced Symptom Analysis

Currently, the symptom checker works on predefined datasets. In future:

1. Larger and more detailed medical datasets can be added
2. Severity-based symptom classification can be introduced
3. Lifestyle-based recommendations can be enhanced

This will improve accuracy and reliability of health guidance.

8.2.4 Multi-Language Support

To increase accessibility across different regions, DOCMEET can support multiple regional languages such as:

1. Hindi
2. Tamil
3. Telugu
4. Marathi

This will help users from rural and non-English-speaking backgrounds use the platform comfortably.

8.2.5 Appointment Reminder System

Automated reminder features can be implemented, including:

1. Email notifications
2. SMS reminders
3. In-app alerts

This helps reduce missed appointments and improves scheduling efficiency.

8.2.6 Mobile Application Development

A dedicated mobile application for Android and iOS can be developed to:

1. Provide faster access to appointments
2. Enable push notifications
3. Improve user engagement

This will increase adoption among smartphone users.

8.2.7 Hospital and Pharmacy Integration

DOCMEET can be expanded by integrating:

1. Hospital management systems
2. Local pharmacies for medicine availability
3. Diagnostic labs for test bookings

This creates an end-to-end healthcare service ecosystem.

8.2.8 Insurance and Billing Integration

Future enhancements may include:

1. Health insurance verification
2. Digital invoice generation
3. Claim history tracking

This will make the platform suitable for large-scale healthcare providers.

8.3 Future Collaboration Possibilities

Stakeholder	Proposed Integration
Hospitals & Clinics	Centralized appointment and patient management
Diagnostic Labs	Online test booking and report upload
Pharmacies	Medicine availability and delivery
Government Health Programs	Digital health record management
NGOs	Healthcare access for rural and underserved areas

8.4 Scope for Industry-Level Deployment

- **Hospital-Level Deployment:** Appointment and doctor schedule management
- **Telemedicine Platforms:** Virtual healthcare services
- **Healthcare Startups:** Scalable SaaS healthcare solution
- **Rural Health Initiatives:** Remote appointment booking and awareness

8.5 Limitations to Address

Current Limitation	Proposed Enhancement
Limited disease dataset	Expand medical datasets
No video consultation	Add real-time video calling
Internet dependency	Offline appointment request feature
Single language support	Multilingual interface

8.6 Long-Term Vision

The long-term vision of DOCMEET is:

"To create a reliable, accessible, and user-friendly digital healthcare platform that connects patients and doctors efficiently."

Key goals include:

- Improving access to healthcare services
- Reducing appointment wait times
- Promoting digital health awareness
- Supporting scalable healthcare solutions

OUTCOME – Screenshots

Screenshot 1: Home Page Interface

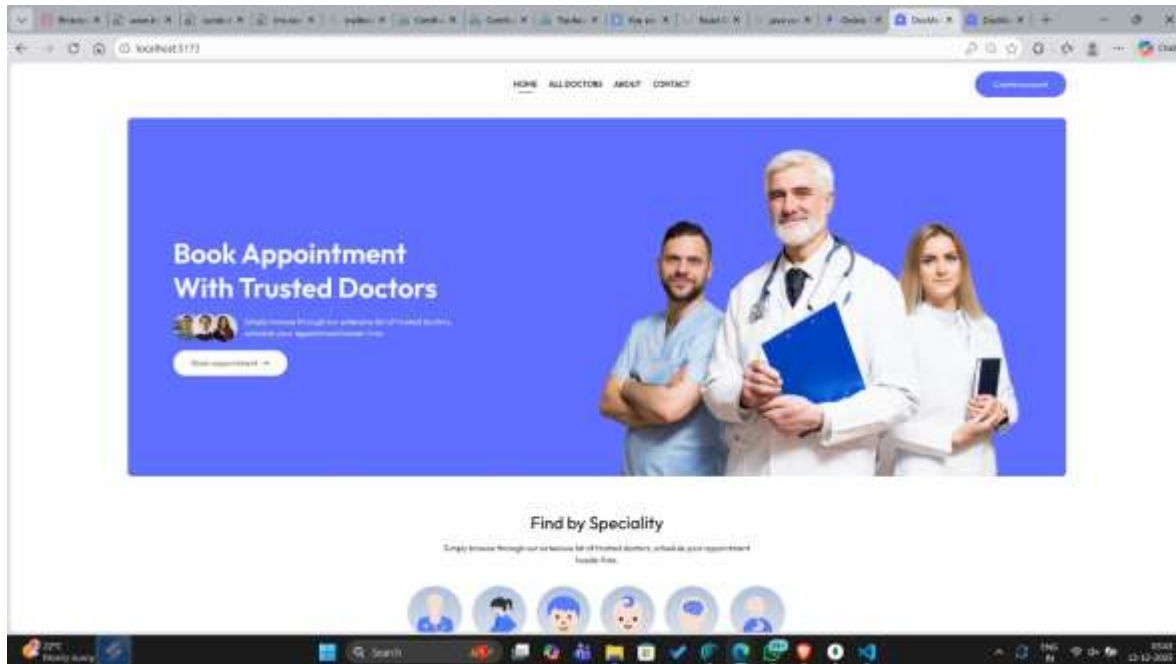


Fig 1 Home page :This is landing page of docmeet.

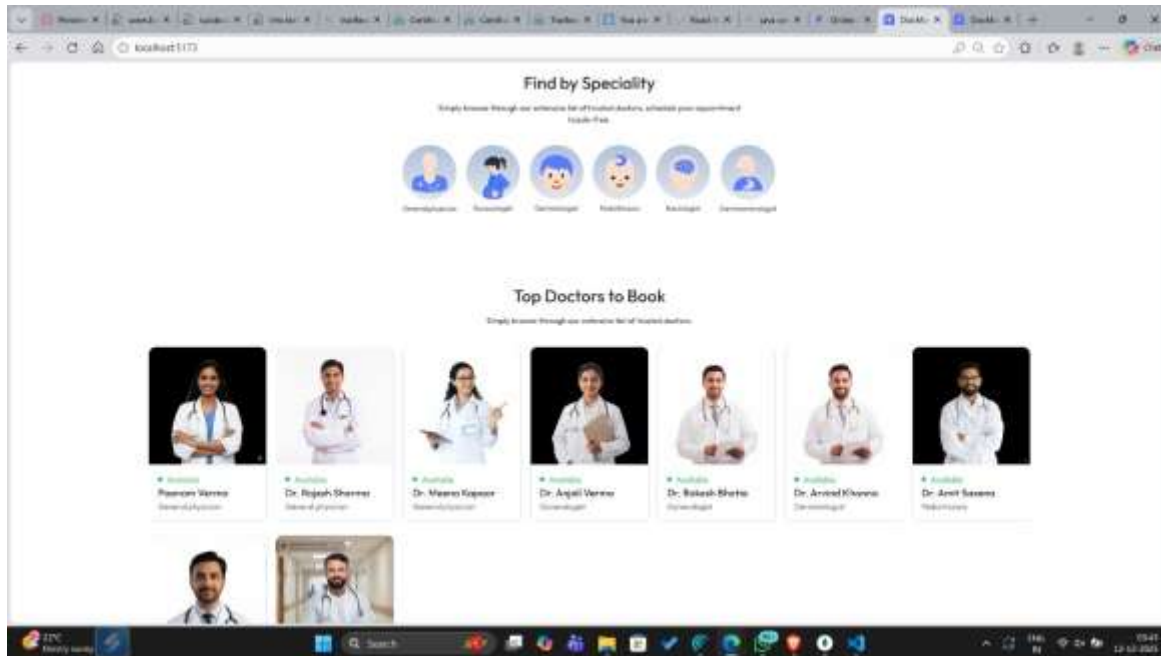


Fig 1.2 Home page

Fig 1.2 shows the home page of the Docmeet which shows details about all the doctors categories which are there and some of the top doctors are listed according to the ratings of the doctors.

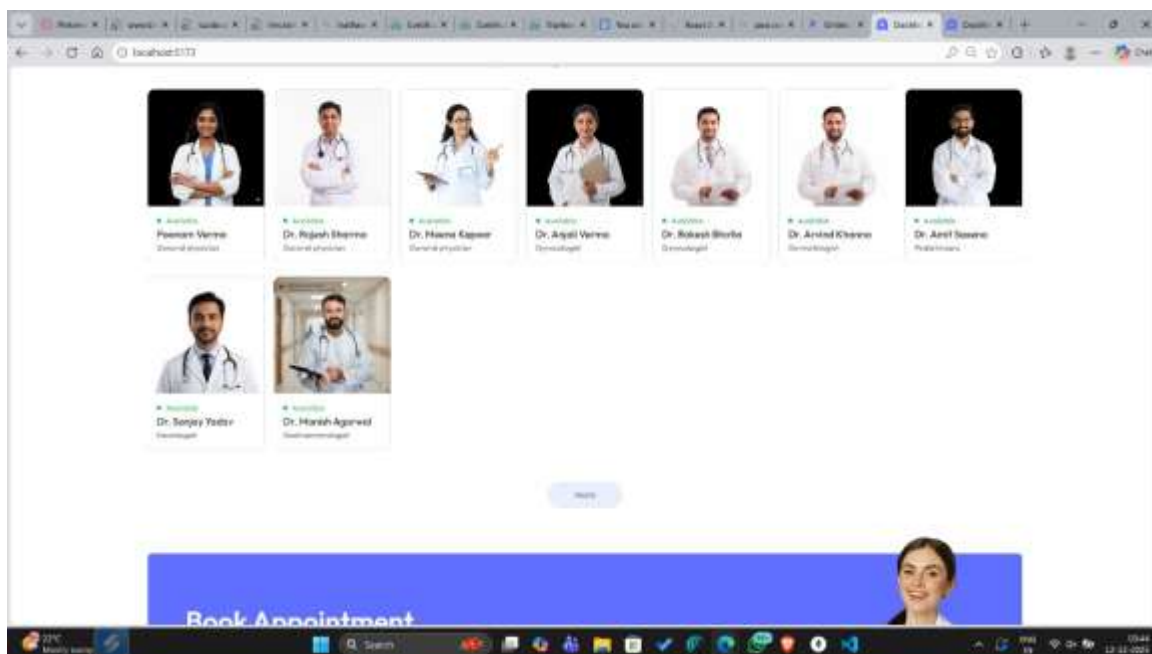


Fig.1.3 Landing home page

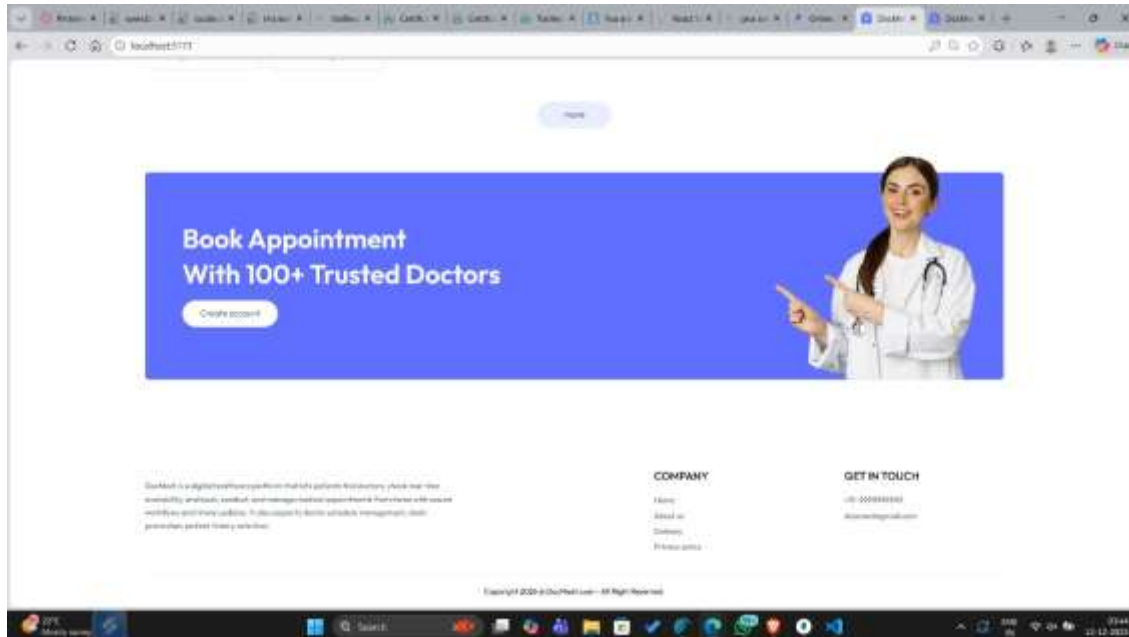


Fig. 1.4 Footer page of homepage of our Docmeet website.

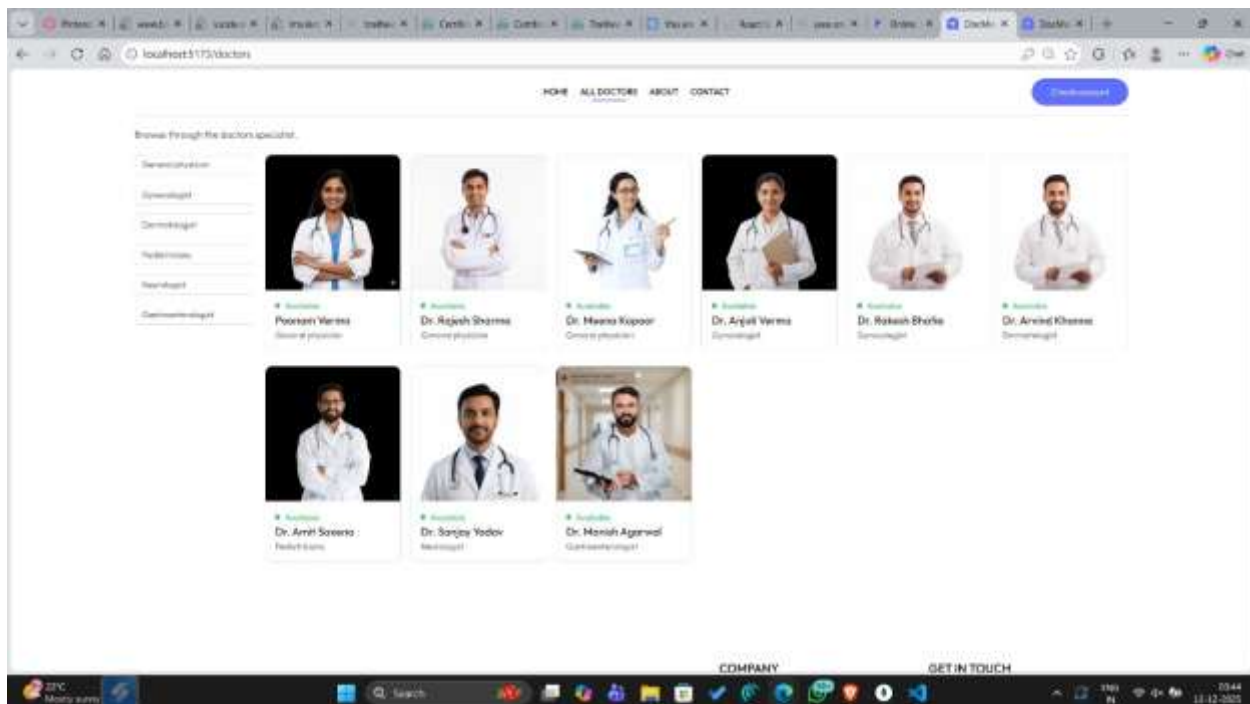


Fig. 1.5 All doctors list page of our Docmeet website.

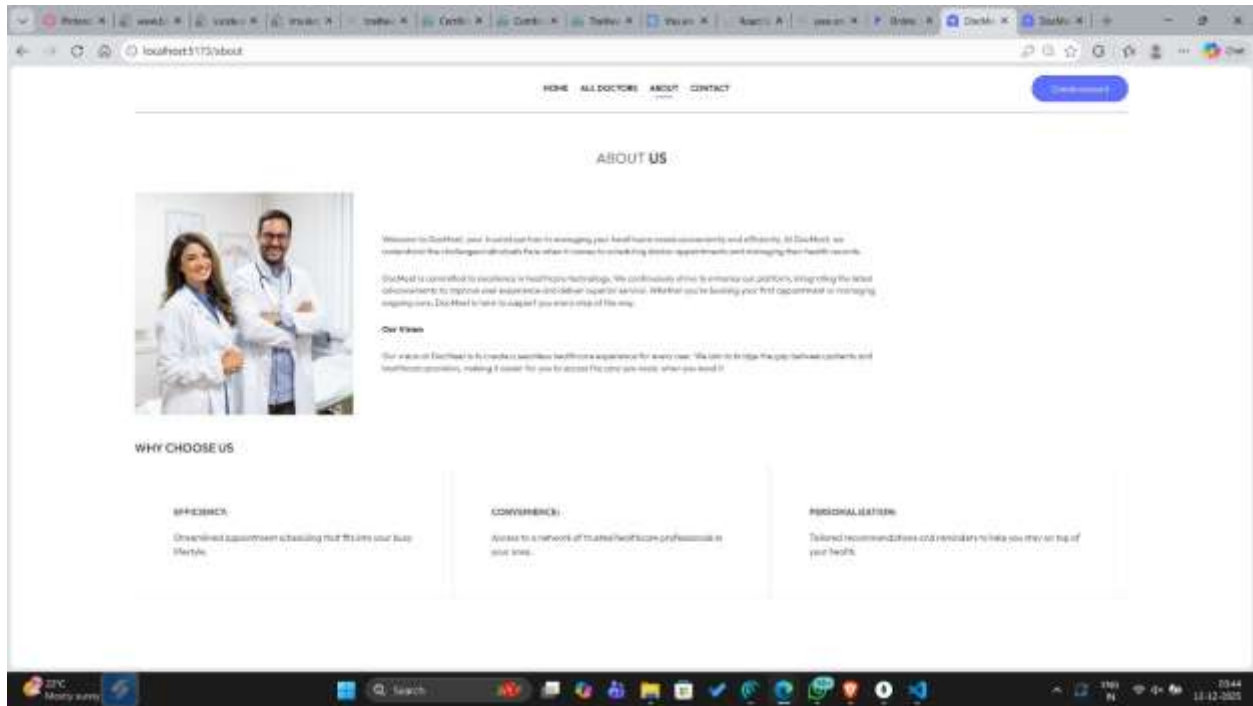


Fig. 1.6 About page of our Docmeet website.

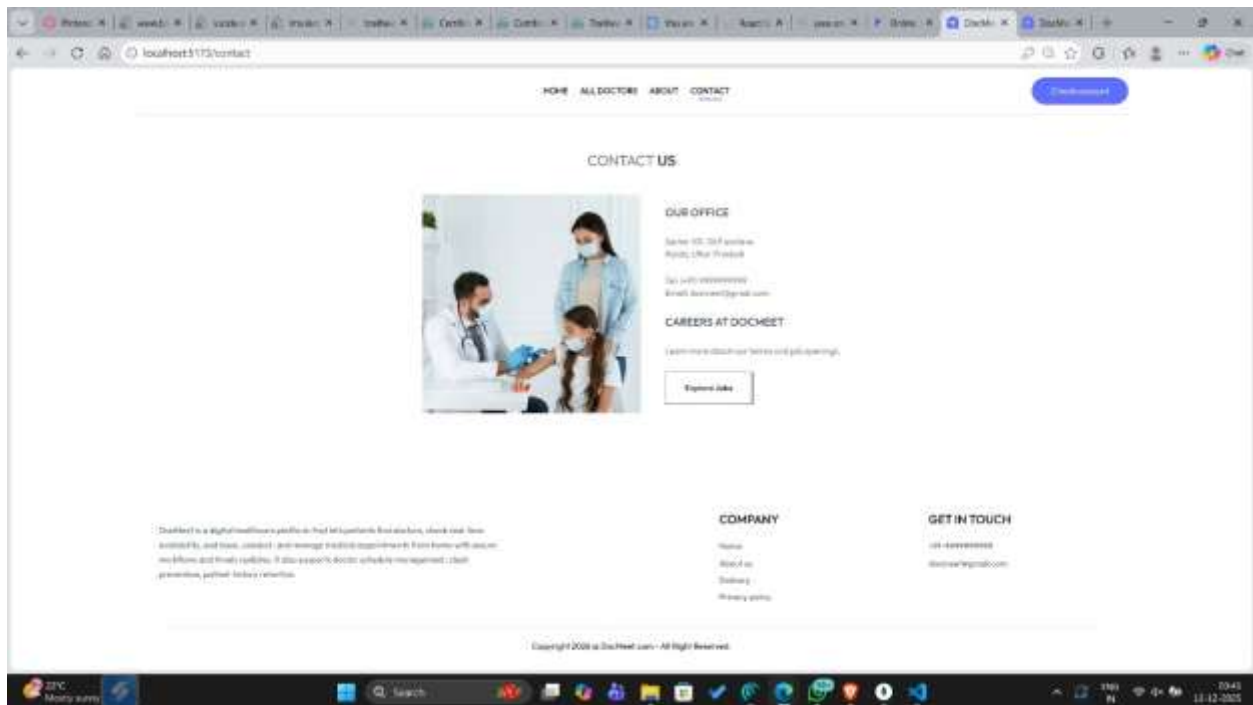


Fig. 1.7 Contact page of our Docmeet website.

Now, here user will login in to our docmeet website for profile creation in order to do appointments to the related doctors.

Screenshot 2: Login and Registration Page

Fields for name, email, and password

Error messages for invalid inputs

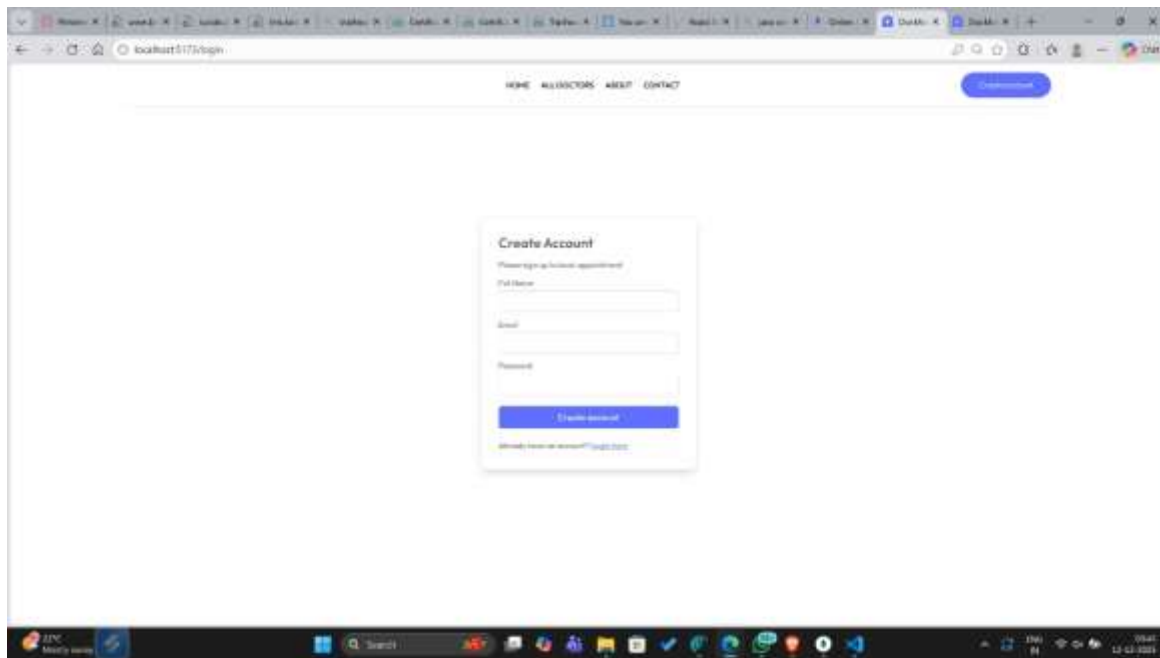


Fig. 2.1 Sign up page for profile creation of our Docmeet website.

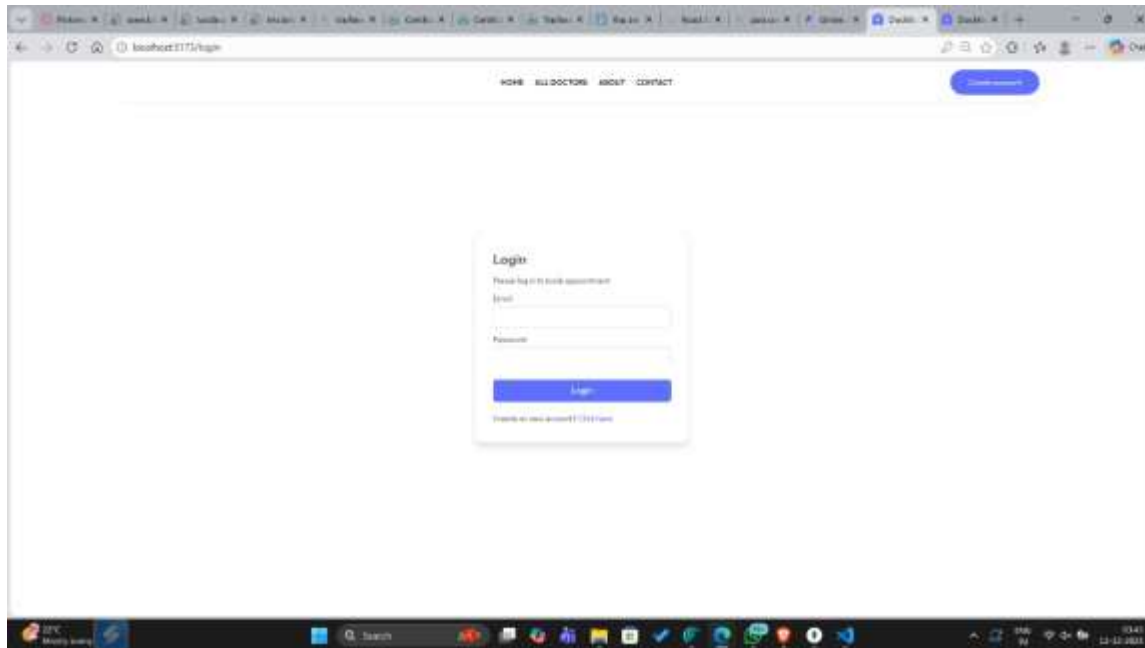


Fig. 2.2 Login page for user on our Docmeet website.

Screenshot 3: Admin Panel

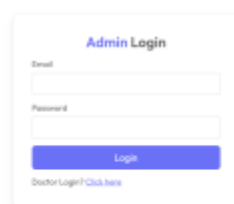


Fig. 3.1 Admin Login page of our Docmeet website.

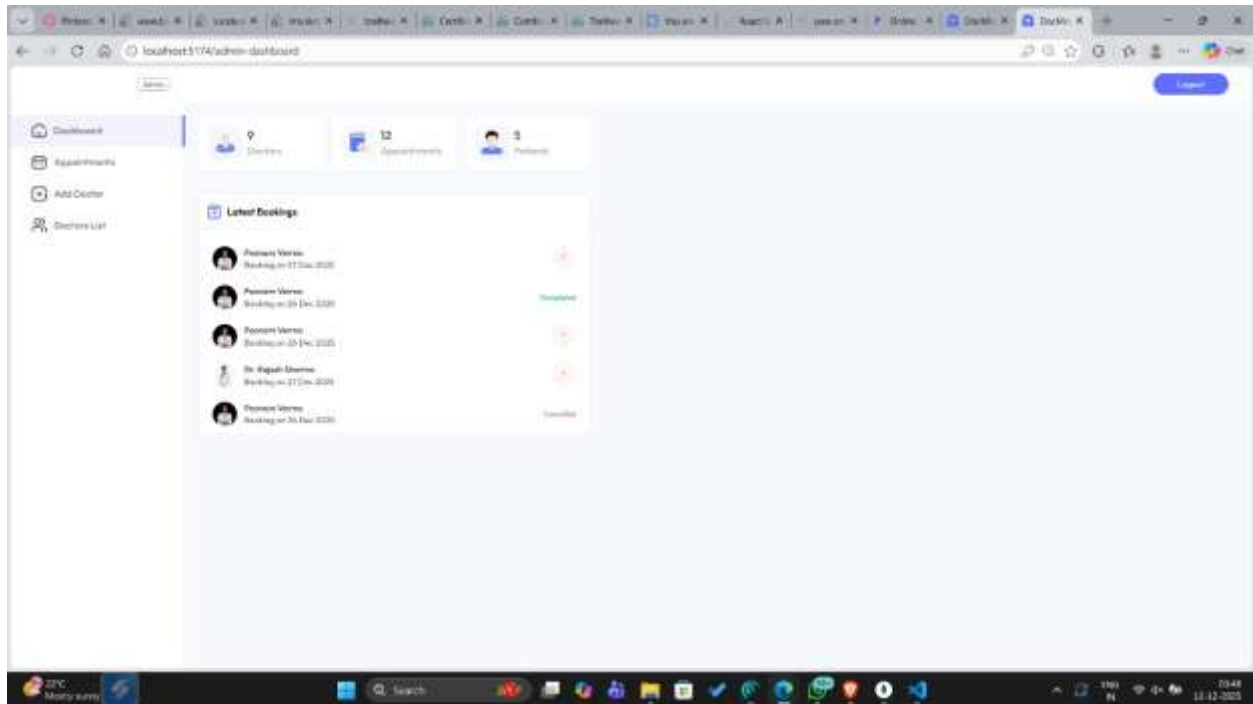


Fig. 3.2 Admin Dashboard after login on website.

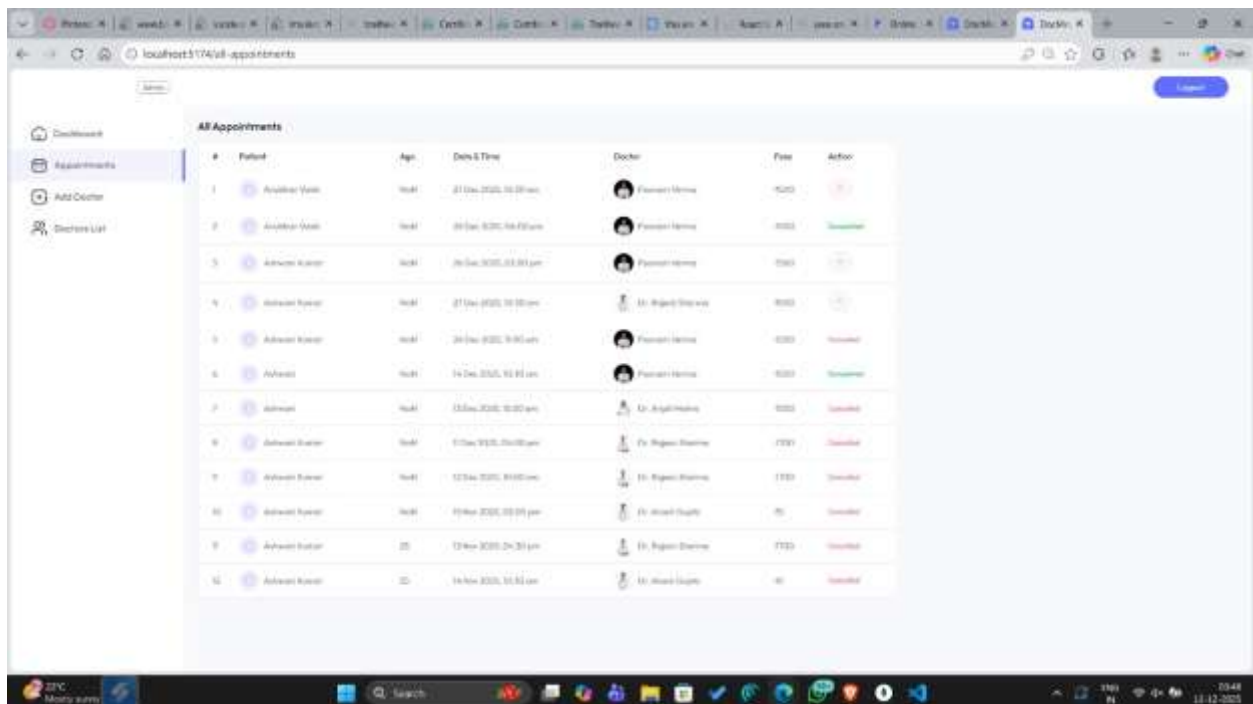


Fig. 3.2 Admins appointment approval or decline page.

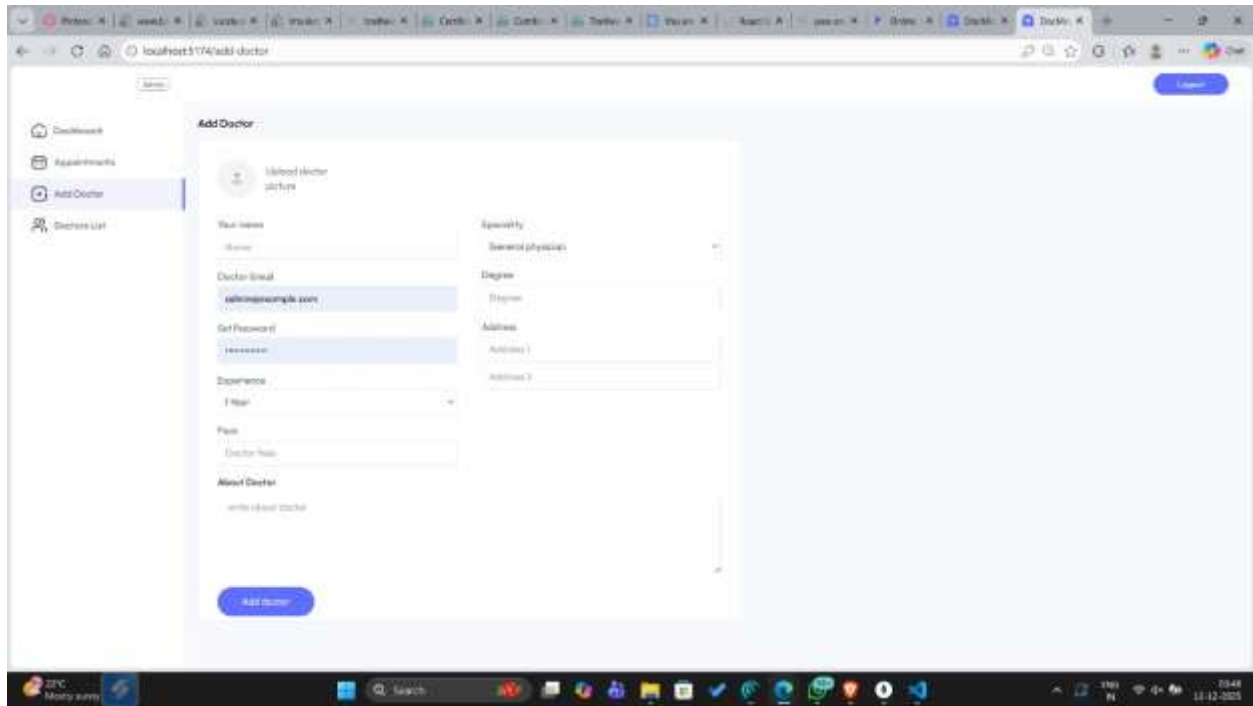


Fig. 3.3 Admins adding and creating new doctor's profile page.

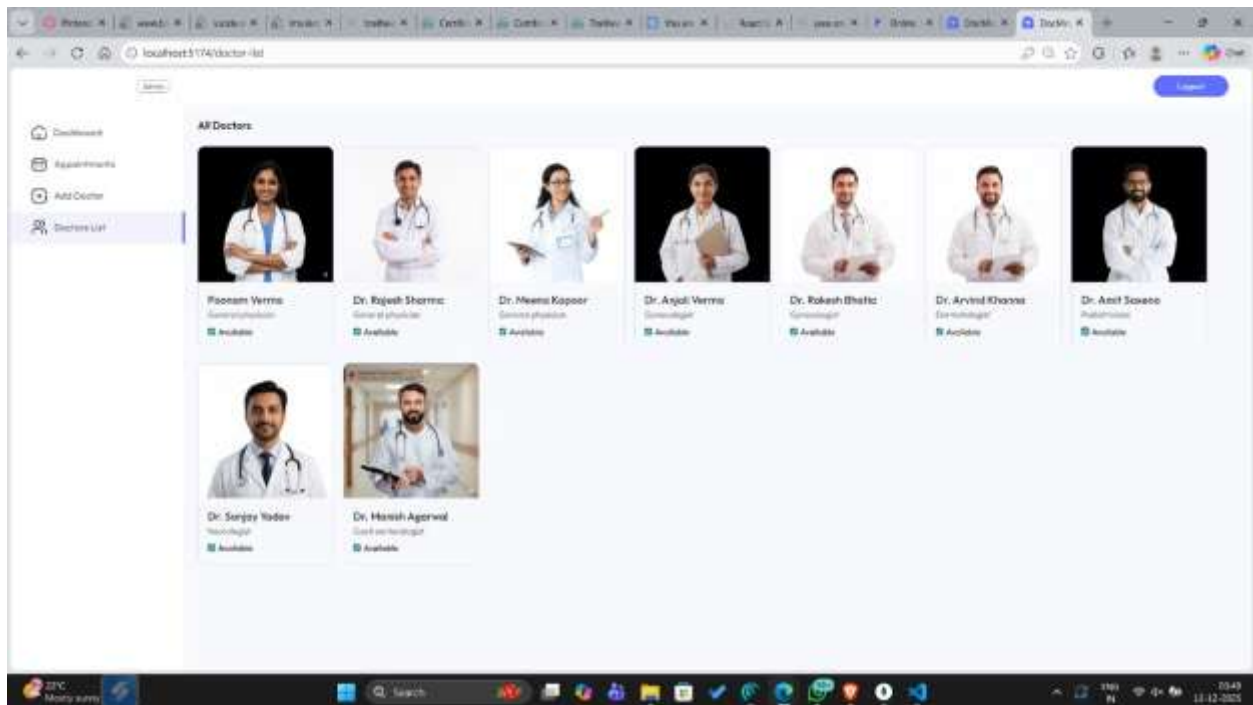


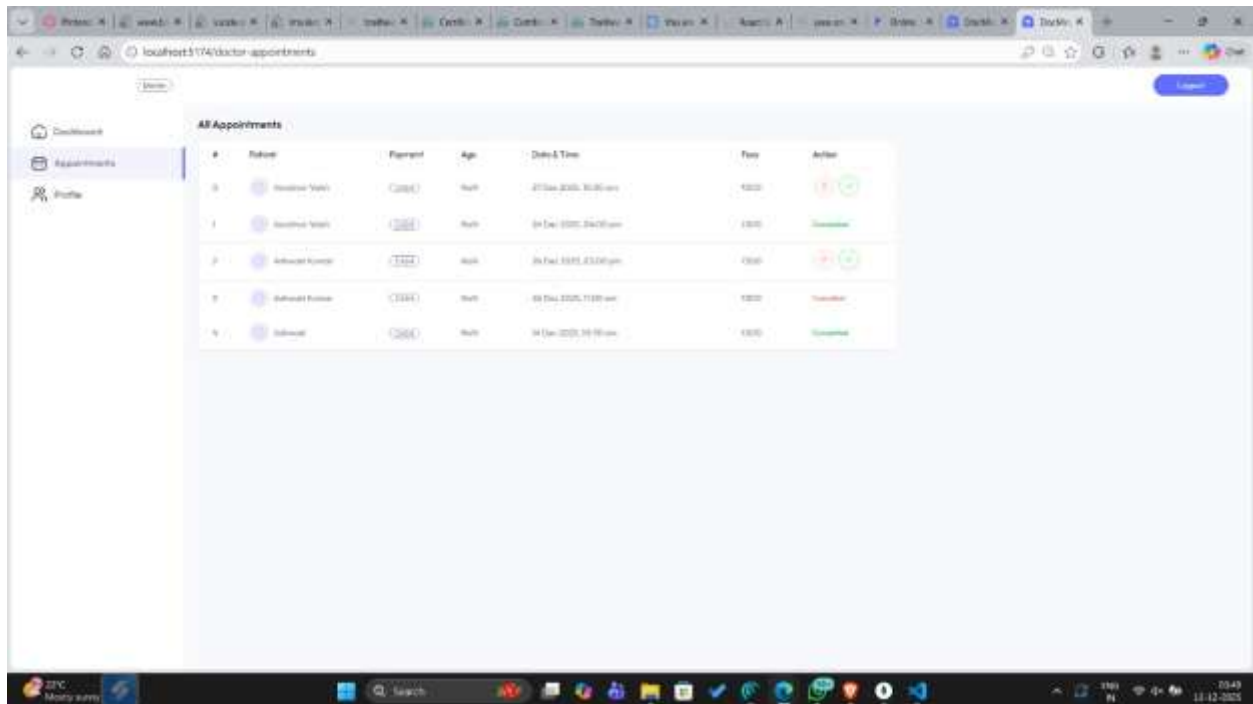
Fig. 3.4 All doctors list inside admin panel page.

Screenshot 4: Doctors Panel



A screenshot of a 'Doctor Login' form. It features a title 'Doctor Login' in blue. Below the title are two input fields: 'Email' and 'Password'. A blue 'Login' button is positioned below the password field. At the bottom, there is a link that says 'Admin Login? Click here'.

Fig. 4.1 Doctors Login page.



A screenshot of a web application interface for managing appointments. The browser address bar shows 'localhost:5174/doctor-appointments'. On the left, a sidebar contains 'Dashboard', 'Appointments' (selected), and 'Profile'. The main area is titled 'All Appointments' and contains a table with the following data:

#	Patient	Payment	Age	Date & Time	Fee	Action
3	Reemeh Yash	Cash	Male	27 Dec 2025, 10:00 am	1000	Cancel Confirm
4	Reemeh Yash	Cash	Male	28 Dec 2025, 10:00 am	1000	Cancel Confirm
5	Reemeh Yash	Cash	Male	29 Dec 2025, 10:00 am	1000	Cancel Confirm
6	Reemeh Yash	Cash	Male	30 Dec 2025, 10:00 am	1000	Cancel Confirm
7	Reemeh Yash	Cash	Male	31 Dec 2025, 10:00 am	1000	Cancel Confirm
8	Reemeh Yash	Cash	Male	01 Jan 2026, 10:00 am	1000	Cancel Confirm

Fig. 4.2 Doctors all users appointments managing page.

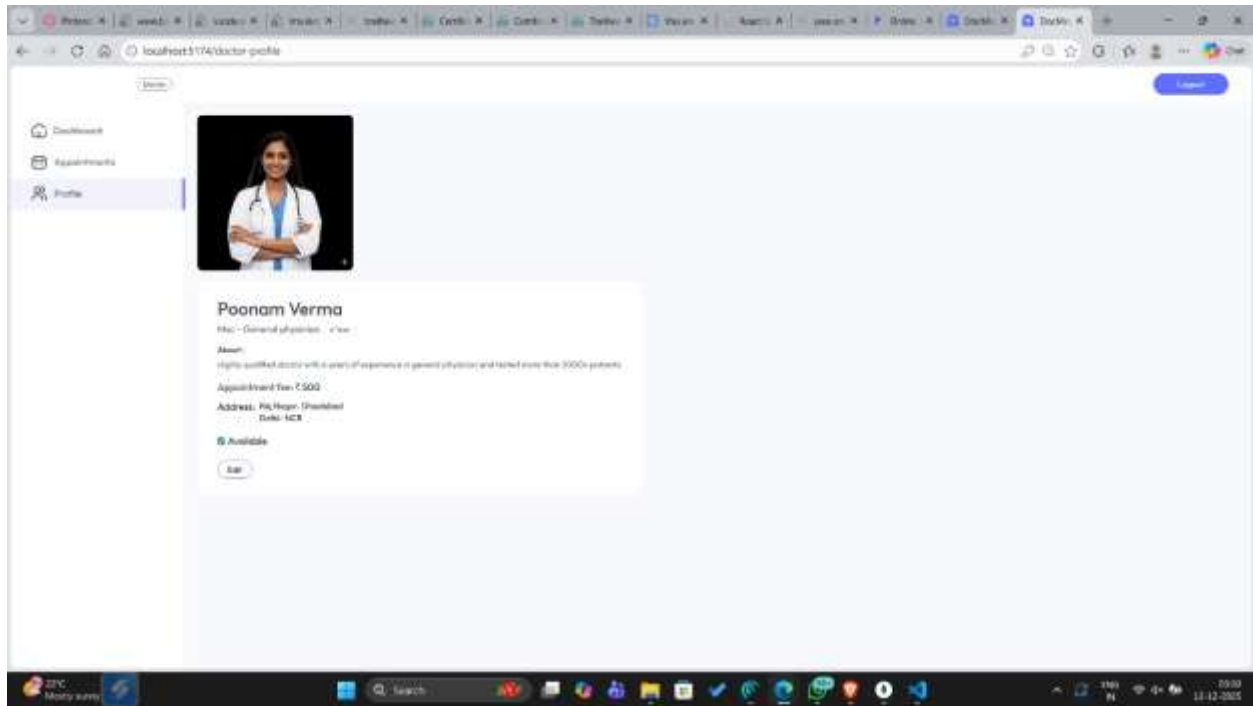


Fig. 4.3 Doctors profile detail editing page.

Screenshot 5: Symptom checker

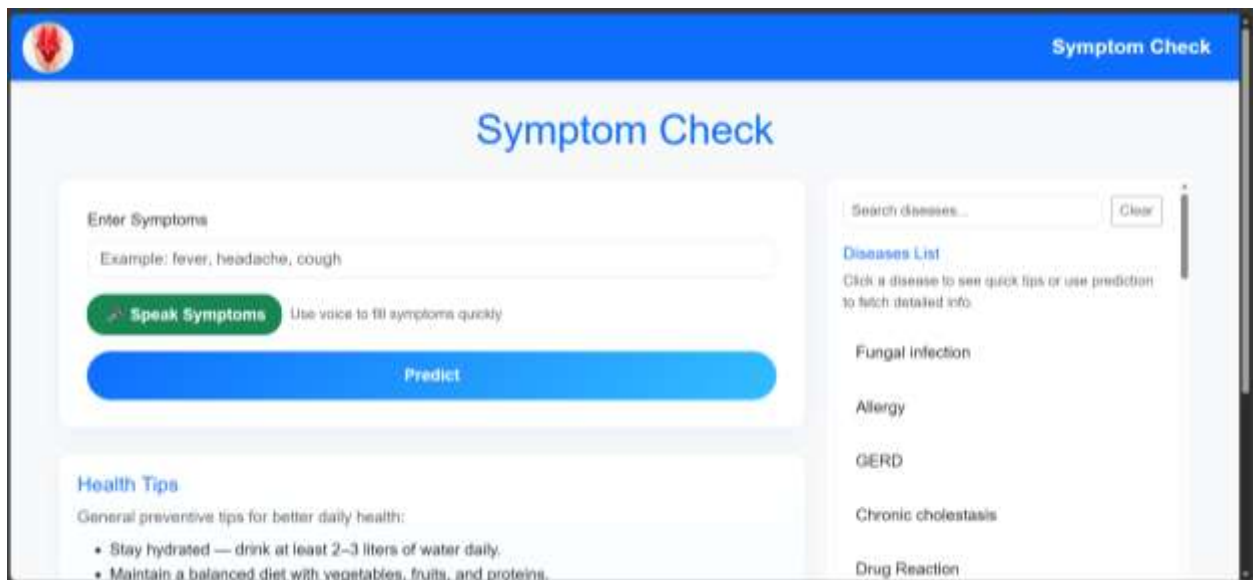


Fig. 5.1 Symptom checker page.

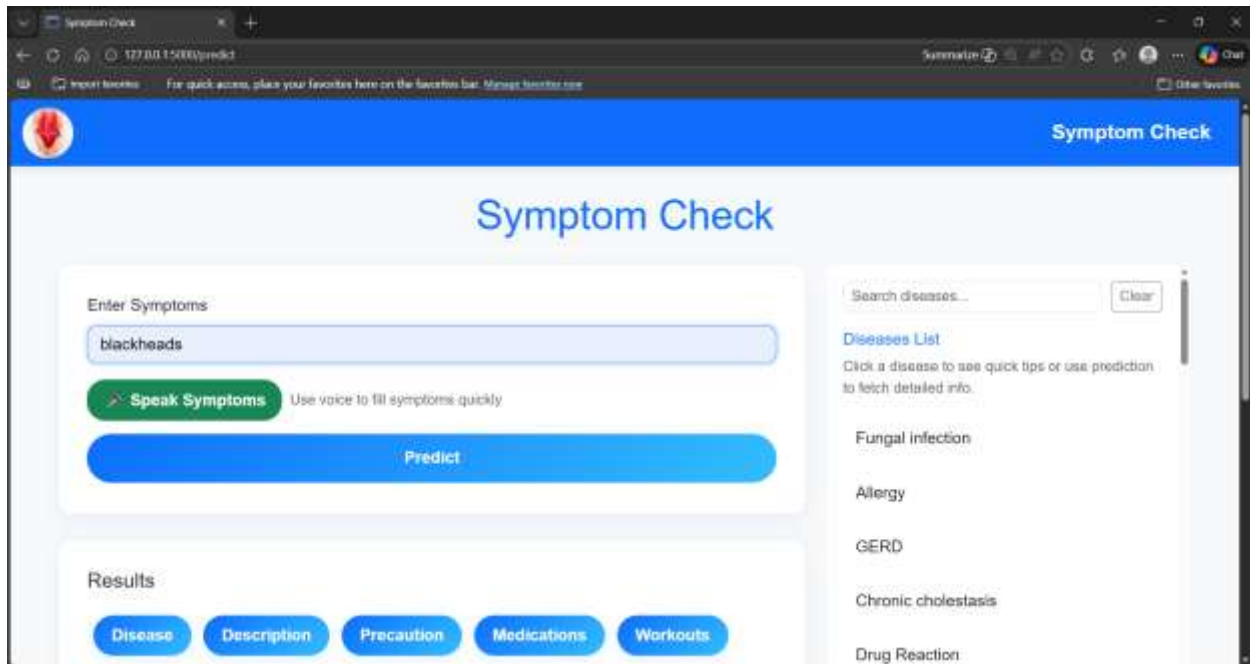


Fig. 5.2 Searching for blackheads symptom .

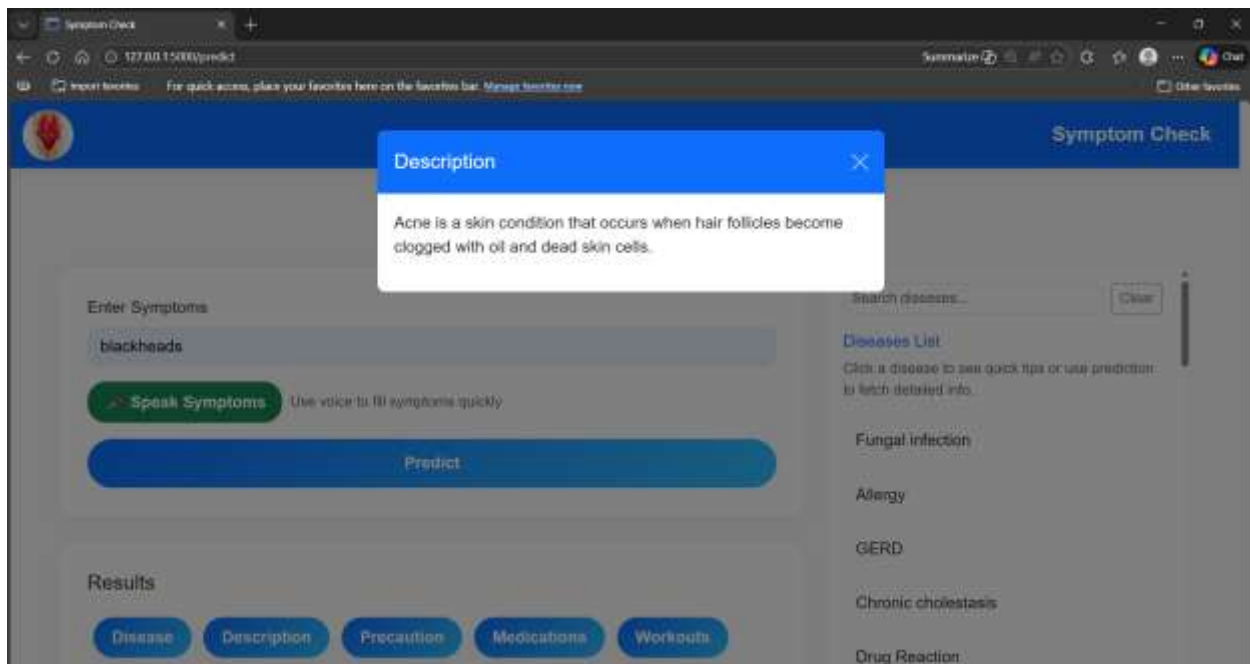


Fig. 5.3 Result shown for Blackheads .This is description shown for blackheads.

Chapter 9

Conclusion

Overview

In today's fast-paced digital world, technology has become a vital enabler in improving accessibility and efficiency across various domains, especially healthcare. The DOCMEET project represents a step toward modernizing the healthcare appointment process by providing a centralized, web-based platform for patients, doctors, and administrators.

DOCMEET simplifies the process of booking doctor appointments, managing schedules, and accessing basic health-related guidance through a symptom checker. By integrating digital appointment scheduling and online payment facilities, the system reduces manual effort, waiting time, and administrative overhead while improving user convenience.

This project demonstrates how web technologies can be effectively used to build a scalable, reliable, and user-friendly healthcare management solution.

9.1 Recap of Problem and Solution

Problem Identified

1. Difficulty in booking doctor appointments through traditional offline methods
2. Long waiting times and inefficient appointment management
3. Lack of a centralized platform for managing doctors, users, and appointments
4. Limited access to basic health guidance for users
5. Manual payment processes causing inconvenience

Proposed Solution

1. Development of a web-based doctor appointment system
2. Role-based access for Admin, Doctor, and User
3. Digital appointment booking with time-slot selection
4. Symptom checker for basic disease information and guidance
5. Online payment integration for confirmed appointments
6. Centralized data storage using MongoDB

9.2 System Reliability and User Feedback Considerations

The DOCMEET system has been developed with reliability and usability as primary goals. The application includes:

- Input validation to prevent incorrect data entry
- Secure authentication and role-based authorization
- Proper error handling for failed transactions or invalid requests
- Structured database design to ensure data consistency

User feedback during testing indicated that the system is easy to navigate and significantly reduces the effort required to book appointments. Future improvements can include user feedback forms and rating systems to further enhance service quality.

9.3 Societal and Healthcare Impact

The broader impact of DOCMEET can be observed in several areas:

- Improved access to healthcare services
- Reduced patient waiting time
- Better organization of doctor schedules
- Increased health awareness through symptom checking
- Support for remote and semi-urban users

The project aligns with sustainable development goals such as:

1. **Good Health and Well-Being**
2. **Reduced Inequalities**
3. **Industry, Innovation, and Infrastructure**

9.4 Technical Enhancement Possibilities

Module	Future Enhancement
Appointment System	Video consultation support
Symptom Checker	Expanded medical datasets
Payment Module	Insurance and billing integration
User Interface	Mobile application support
Notifications	Email and SMS reminders

9.5 Technical Reflection and Knowledge Acquired

Through the development of DOCMEET, the team gained practical experience in:

1. Building full-stack applications using the MERN architecture
2. Designing RESTful APIs for healthcare systems
3. Managing databases using MongoDB
4. Implementing secure authentication and authorization
5. Integrating payment gateways
6. Handling real-world user workflows and validations

Tools and technologies used during development include:

- React.js
- Node.js and Express.js
- MongoDB
- Git and GitHub
- VS Code
- Postman

9.6 Entrepreneurial and Business Potential

DOCMEET has strong potential to evolve into a commercial healthcare solution. Possible deployment models include:

1. Subscription-based service for clinics and hospitals
2. SaaS platform for healthcare startups
3. White-labeled solutions for private hospitals
4. Integration with telemedicine platforms

9.7 Closing Perspective

DOCMEET is more than just a final-year academic project—it is a practical healthcare solution addressing real-world challenges. By combining appointment scheduling, basic health guidance, and digital payments into a single platform, DOCMEET enhances both patient experience and healthcare management efficiency.

Final Summary

“DOCMEET provides a simple, reliable, and accessible digital platform that bridges the gap between patients and healthcare providers, making healthcare services more organized and convenient.”

Chapter 10

References

10.1 Books and Academic Resources

1. **“Web Technologies: HTML, JavaScript, PHP, Java, JSP, ASP.NET, XML and AJAX – Black Book”**

Author: Kogent Learning Solutions Inc.

Publisher: Dreamtech Press

Used For: Understanding frontend and backend web development concepts.

2. **“MongoDB: The Definitive Guide”**

Author: Kristina Chodorow

Publisher: O’Reilly Media

Used For: Database design, schema modeling, and CRUD operations using MongoDB.

3. **“Learning React”**

Authors: Alex Banks, Eve Porcello

Publisher: O’Reilly Media

Used For: Building component-based user interfaces and managing state.

4. **“Node.js Design Patterns”**

Author: Mario Casciaro

Publisher: Packt Publishing

Used For: Backend architecture, API development, and scalability.

10.2 Research Papers and Healthcare Resources

1. **“Web-Based Healthcare Management Systems: A Review”**

Journal: International Journal of Computer Applications

Used For: Understanding online appointment and healthcare systems.

2. **“Design and Implementation of Online Doctor Appointment System”**

Conference: IEEE International Conference on Computing

Used For: System workflow and appointment scheduling logic.

10.3 Websites and Online Resources

1. <https://reactjs.org/>
Used For: React.js documentation, components, hooks, and best practices.
2. <https://nodejs.org/>
Used For: Server-side JavaScript runtime and backend development.
3. <https://expressjs.com/>
Used For: RESTful API development and routing.
4. <https://www.mongodb.com/docs/>
Used For: MongoDB database operations and schema design.
5. <https://developer.mozilla.org/>
Used For: JavaScript syntax, DOM manipulation, and web standards.
6. <https://tailwindcss.com/>
Used For: Responsive UI design and styling utilities.

10.4 Tools and Development Platforms

1. **Visual Studio Code Documentation** – <https://code.visualstudio.com/docs>
Used For: Development environment and debugging.
2. **GitHub Documentation** – <https://docs.github.com>
Used For: Version control and project collaboration.
3. **Postman Documentation** – <https://learning.postman.com>
Used For: API testing and request validation.

10.5 Datasets and Medical References

1. **Public Medical Symptom and Disease Datasets**
Used For: Symptom checker dataset preparation (symptoms, precautions, diet, medication).
2. **World Health Organization (WHO)** – <https://www.who.int>
Used For: General disease descriptions and healthcare standards reference.
3. **MedlinePlus** – <https://medlineplus.gov>
Used For: Basic medical information and disease awareness.